

```
root@kali:~# nmap -sS 172.16.50.20
Starting Nmap 7.99 ( https://nmap.org )
Nmap scan report for 172.16.50.20
Host is up (0.0017s latency).
PORT      STATE SERVICE
135/tcp    open  msrpc
139/tcp    open  netbios-ssn
445/tcp    open  microsoft-ds
3389/tcp   open  ms-wbt-server

Nmap done: 1 IP address (1 host up)
```

```
root@kali:~#
```

```
SCAN
DETECT
ANALYZE
LOG
RESPOND
PROTECT
```

# SOC ANALYST CHECKER

THREAT INTELLIGENCE  
INCIDENT RESPONSE  
SECURITY MONITORING  
LOG ANALYSIS  
NETWORK DEFENSE

Student: Yonatan Menashe  
SOC ANALYST Project - NX220

A SAFER  
CYBER TOMORROW

# Project Overview

The SOC Analyst Checker is a Bash-based tool developed for an authorized training environment. Its purpose is to generate controlled network activity that can be used to validate SOC monitoring and detection processes. The system allows the user to discover available hosts, select a target, choose one of several predefined attack scenarios, execute the activity, and record the result in a structured audit log.

## Main Objectives

- Automate controlled SOC validation activities.
- Discover available hosts on the local network.
- Provide multiple attack scenarios through a simple menu.
- Allow manual or random attack selection.
- Allow manual or random target selection.
- Record each selected activity under /var/log.
- Store timestamps, attack type, target IP, execution status and exit code.
- Keep all testing restricted to an authorized private lab environment.

# System Workflow

The Checker follows a structured workflow from startup to attack execution and logging. Each stage validates the environment and user input before network activity is performed.



At startup, the script verifies that it is running in an authorized lab environment and checks the required system components. It then detects the active network configuration and discovers available hosts. The user selects an attack scenario and target manually or randomly. Before execution, the selected configuration is displayed for review. The activity is then executed and its lifecycle is recorded in the audit log.

# Network & Host Discovery

Before an attack can be executed, the Checker automatically identifies the active network environment and discovers available hosts on the local subnet. This provides the user with valid targets without requiring manual network configuration.

## Network Detection

The script uses the Linux routing configuration to automatically identify:

- Active network interface
- Local IPv4 address
- Local subnet
- Default gateway

## Host Discovery

The Checker uses Nmap to discover active hosts.

```
nmap -sn <subnet>
```

The results are parsed and displayed to the user.

### Network Information

```
Interface : eth1
Local IP   : 172.16.50.69
Network    : 172.16.50.0/24
Gateway    : 172.16.50.1
```

### Discovered hosts:

```
172.16.50.20
172.16.50.69 (local host; not eligible)
```

# Attack Selection

After host discovery, the Checker displays the attack selection menu. Three controlled network scenarios are available, each designed to generate a different type of network activity for SOC analysis.

## Available Scenarios

TCP SYN Scan - TCP port discovery

UDP Scan - UDP port discovery

Service Detection - Service and version identification

## User Controls

Attacks can be selected manually or randomly. The menu also provides Help and Quit options, while invalid input is rejected safely.

```
Select Attack
```

```
[1] TCP SYN Scan
```

```
Limited scan of the 100 most common TCP ports.
```

```
[2] UDP Scan
```

```
Limited scan of the 20 most common UDP ports.
```

```
[3] Service Detection Scan
```

```
Detects services and versions on 20 common TCP ports.
```

```
[R] Random Attack
```

```
[H] Help
```

```
[Q] Quit
```

```
Select attack: █
```

# Target Selection & Execution Summary

After selecting an attack, the Checker presents the discovered eligible targets. The user can select a listed host, enter a private IP manually, or request a random target.

## Pre-Execution Validation

Before any network activity begins, the Checker displays a final execution summary and asks the user for confirmation.

## Safety Controls

- Only private RFC1918 targets allowed
- Local host excluded
- Default gateway excluded from random selection
- Final confirmation required before execution

```
=====
Select Target
=====
Discovered targets:
[1] 172.16.50.20

[M] Enter IP manually
[R] Random target
[Q] Quit

Select target: 1
Selected target: 172.16.50.20

Selected attack: TCP SYN Scan
=====

Execution Summary
=====
Attack      : TCP SYN Scan
Type       : REAL NETWORK ACTIVITY
Target      : 172.16.50.20
Interface   : eth1
Network     : 172.16.50.0/24
Log         : /var/log/soc_checker.log

Description: Performs a limited SYN scan against the 100 most common TCP ports.

Proceed? [y/N] █
```

# TCP SYN Scan

The TCP SYN Scan performs a controlled scan of the 100 most common TCP ports on the selected target.

## Command

```
nmap -Pn -sS --top-ports 100 <target>
```

## Test Result

The scan was successfully executed against 172.16.50.20. Five open TCP ports were identified during the test.

```
Running TCP SYN Scan against 172.16.50.20 ...
Command: nmap -Pn -sS --top-ports 100 172.16.50.20

Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-15 17:32 +0300
Nmap scan report for 172.16.50.20
Host is up (0.0014s latency).
Not shown: 95 closed tcp ports (reset)
PORT      STATE SERVICE
135/tcp    open  msrpc
139/tcp    open  netbios-ssn
445/tcp    open  microsoft-ds
3389/tcp   open  ms-wbt-server
5357/tcp   open  wsdapi
MAC Address: 00:0C:29:2A:3E:4E (VMware)

Nmap done: 1 IP address (1 host up) scanned in 1.88 seconds
[+] Attack completed successfully.
[+] Audit log: /var/log/soc_checker.log
```

# UDP Scan

The UDP Scan performs a controlled scan of the 20 most common UDP ports on the selected target.

## Command

```
nmap -Pn -sU --top-ports 20 <target>
```

## Test Result

The scan was successfully executed against 172.16.50.20. The results identified UDP port 137 as open and several additional ports as open|filtered.

```
Running UDP Scan against 172.16.50.20...
Command: nmap -Pn -sU --top-ports 20 172.16.50.20

Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-15 17:48 +0300
Nmap scan report for 172.16.50.20
Host is up (0.0067s latency).

PORT      STATE      SERVICE
53/udp    open|filtered domain
67/udp    open|filtered dhcp
68/udp    closed     dhcp
69/udp    closed     tftp
123/udp   open|filtered ntp
135/udp   closed     msrpc
137/udp   open       netbios-ns
138/udp   open|filtered netbios-dgm
139/udp   closed     netbios-ssn
161/udp   open|filtered snmp
162/udp   closed     snmptrap
445/udp   open|filtered microsoft-ds
500/udp   open|filtered isakmp
514/udp   closed     syslog
520/udp   closed     route
631/udp   closed     ipp
1434/udp  closed     ms-sql-m
1900/udp  open|filtered upnp
4500/udp  open|filtered nat-t-ike
49152/udp closed     unknown
MAC Address: 00:0C:29:2A:3E:4E (VMware)

Nmap done: 1 IP address (1 host up) scanned in 8.30 seconds
[+] Attack completed successfully.
[+] Audit log: /var/log/soc_checker.log
```

# Service Detection Scan

The Service Detection Scan analyzes 20 common TCP ports and attempts to identify the services and software running on the selected target.

## Command

```
nmap -Pn -sV --top-ports 20 <target>
```

## Test Result

The scan was successfully executed against 172.16.50.20. Four open TCP ports were identified, and Nmap detected multiple Microsoft Windows services.

```
Running Service Detection Scan against 172.16.50.20 ...
Command: nmap -Pn -sV --top-ports 20 172.16.50.20

Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-15 17:54 +0300
Nmap scan report for 172.16.50.20
Host is up (0.00066s latency).

PORT      STATE SERVICE      VERSION
21/tcp    closed ftp
22/tcp    closed ssh
23/tcp    closed telnet
25/tcp    closed smtp
53/tcp    closed domain
80/tcp    closed http
110/tcp   closed pop3
111/tcp   closed rpcbind
135/tcp   open  msrpc        Microsoft Windows RPC
139/tcp   open  netbios-ssn  Microsoft Windows netbios-ssn
143/tcp   closed imap
443/tcp   closed https
445/tcp   open  microsoft-ds?
993/tcp   closed imaps
995/tcp   closed pop3s
1723/tcp  closed pptp
3306/tcp  closed mysql
3389/tcp  open  ms-wbt-server Microsoft Terminal Services
5900/tcp  closed vnc
8080/tcp  closed http-proxy
MAC Address: 00:0C:29:2A:3E:4E (VMware)
Service Info: OS: Windows; CPE: cpe:/o:microsoft:windows

Service detection performed. Please report any incorrect results at
Nmap done: 1 IP address (1 host up) scanned in 17.54 seconds
[+] Attack completed successfully.
[+] Audit log: /var/log/soc_checker.log
```

# Audit Logging

Every attack executed by the Checker is recorded in `/var/log/soc_checker.log`, creating a structured and traceable record of each test session.

Each entry includes the session ID, timestamp, attack type, target IP, execution status and exit code. The attack lifecycle is tracked through SELECTED, STARTED and COMPLETED or FAILED.

```
(root@kali)-[/home/kali]
# sudo tail -n 15 /var/log/soc_checker.log
2026-09-15T17:01:49+03:00 | SESSION=20260915_170148_7137 | EVENT=PROGRAM_START
2026-09-15T17:29:29+03:00 | SESSION=20260915_170148_7137 | ATTACK=TCP_SYN_SCAN | TARGET=172.16.50.20 | STATUS=SELECTED
2026-09-15T17:32:26+03:00 | SESSION=20260915_170148_7137 | ATTACK=TCP_SYN_SCAN | TARGET=172.16.50.20 | STATUS=STARTED
2026-09-15T17:32:28+03:00 | SESSION=20260915_170148_7137 | ATTACK=TCP_SYN_SCAN | TARGET=172.16.50.20 | STATUS=COMPLETED | EXIT_CODE=0
2026-09-15T17:32:28+03:00 | SESSION=20260915_170148_7137 | EVENT=PROGRAM_END | EXIT_CODE=0
2026-09-15T17:47:46+03:00 | SESSION=20260915_174744_30629 | EVENT=PROGRAM_START
2026-09-15T17:47:56+03:00 | SESSION=20260915_174744_30629 | ATTACK=UDP_SCAN | TARGET=172.16.50.20 | STATUS=SELECTED
2026-09-15T17:48:00+03:00 | SESSION=20260915_174744_30629 | ATTACK=UDP_SCAN | TARGET=172.16.50.20 | STATUS=STARTED
2026-09-15T17:48:09+03:00 | SESSION=20260915_174744_30629 | ATTACK=UDP_SCAN | TARGET=172.16.50.20 | STATUS=COMPLETED | EXIT_CODE=0
2026-09-15T17:48:09+03:00 | SESSION=20260915_174744_30629 | EVENT=PROGRAM_END | EXIT_CODE=0
2026-09-15T17:54:17+03:00 | SESSION=20260915_175415_34012 | EVENT=PROGRAM_START
2026-09-15T17:54:29+03:00 | SESSION=20260915_175415_34012 | ATTACK=SERVICE_DETECTION | TARGET=172.16.50.20 | STATUS=SELECTED
2026-09-15T17:54:34+03:00 | SESSION=20260915_175415_34012 | ATTACK=SERVICE_DETECTION | TARGET=172.16.50.20 | STATUS=STARTED
2026-09-15T17:54:51+03:00 | SESSION=20260915_175415_34012 | ATTACK=SERVICE_DETECTION | TARGET=172.16.50.20 | STATUS=COMPLETED | EXIT_CODE=0
2026-09-15T17:54:51+03:00 | SESSION=20260915_175415_34012 | EVENT=PROGRAM_END | EXIT_CODE=0
```

# Testing Environment & Results

## Lab Environment

The Checker was tested in an isolated private lab environment using Kali Linux as the testing system and a Windows machine as the target.

**Kali Linux:** 172.16.50.69

**Windows Target:** 172.16.50.20

**Network:** 172.16.50.0/24

**Interface:** eth1

## Test Results

**Network & Host Discovery - PASSED**

**TCP SYN Scan - PASSED**

**UDP Scan - PASSED**

**Service Detection Scan - PASSED**

**Audit Logging - VERIFIED**

## Final Validation

All three network scenarios were successfully executed against the Windows lab target. Host discovery, target selection, attack execution and audit logging operated as expected throughout the testing process.

# Conclusion

The SOC Analyst Checker was successfully developed and tested as an automated Bash-based tool for controlled SOC validation activities in an authorized lab environment.

The project combines automatic network discovery, attack and target selection, input validation, controlled Nmap-based network activity and structured audit logging. All three implemented scenarios, including TCP SYN Scan, UDP Scan and Service Detection Scan, were successfully executed against the Windows lab target.

Testing confirmed that the Checker correctly discovers available hosts, validates eligible targets, executes the selected network activity and records the complete attack lifecycle in `/var/log/soc_checker.log`.

## Project Outcome

The final system provides a simple and repeatable method for generating controlled network activity that can be used to support SOC monitoring, detection and validation exercises.